

Git et GitHub : mémo de travail

ENSUP Business School · Git Bash · Les noms en MAJUSCULES sont à remplacer

Pour cette journée : cloner votre fork, puis travailler dans `rendus/IDENTIFIANT`. `origin` désigne votre fork ; `upstream` désigne le dépôt `AbidHamza/ensup-git-github-2026`. Le clone contient déjà l'historique initial.

Le cycle local

```
git status          # Observer l'état
git diff            # Travail vs index
git add README.md   # Préparer ce fichier
git diff --cached    # Index vs dernier commit
git commit -m "docs: préciser le but"
git log --oneline    # Lire l'historique
git show HEAD        # Voir le dernier commit
```

Enregistrer dans l'éditeur ≠ préparer avec `add` ≠ commiter ≠ publier avec `push`.

Démarrer

```
git init -b main    # Dans un dossier neuf
git config user.name "Prenom Nom" # Identité locale
git config user.email "ADRESSE"
git clone URL_DU_DEPOT # Autre cas : dépôt existant
```

Choisir **init** pour démarrer un dépôt local, ou **clone** pour récupérer un dépôt existant. Ne pas lancer les deux comme une recette unique.

Travailler sur une branche

```
git branch
git switch -c feature/ma-tache
git switch main
git merge feature/ma-tache # main reçoit la branche
git log --graph --oneline --all
```

Dans les exercices, partir d'un état propre avant de changer de branche. Une fusion `fast-forward` n'a pas besoin de créer un nouveau commit.

Partager

```
git remote -v
git remote add origin URL_DU_DEPOT # Une fois, si origin absent
git push -u origin main             # Premier envoi de main
git push -u origin feature/ma-tache # Premier envoi de la branche
git fetch origin                    # Récupérer sans intégrer
git switch main
git pull --ff-only                  # Mettre à jour sans divergence
```

Si `pull --ff-only` refuse, inspecter l'historique avec le formateur. Ne pas répondre par un `push` forcé.

Collaborer sur GitHub

Branche → commits → push → PR vers main → revue → correction → fusion → pull sur chaque PC. Dans la PR, **base reçoit compare**. Pour la PR de travail, base et head repository sont votre fork. Pour la PR de collecte, base est le dépôt AbidHamza et head votre fork, tous deux sur main. Garder cette PR ouverte et pousser les nouveaux commits sur main actualise le rendu. Choisir Create a merge commit pour conserver les commits.

Réparer avec intention

```
git restore --staged FICHIER # Retirer de l'index, garder le travail
git restore FICHIER         # Jeter les écarts travail/index
git revert --no-edit HEAD    # Inverser le dernier commit simple
git merge --abort            # Abandonner une fusion en cours
```

Avant restore FICHIER, relire diff : les changements non préparés seront remplacés. revert ajoute un commit ; il peut demander une résolution de conflit. Pour la désindexation ci-dessus, un commit initial doit déjà exister.

Résoudre un conflit

1. Lire status et ouvrir le fichier.
2. Comprendre les deux propositions, convenir du résultat.
3. Retirer tous les marqueurs et enregistrer le contenu voulu.
4. git add FICHIER, puis git commit -m "fix: résoudre le conflit".
5. Vérifier le résultat, status et le graphe ; pousser si nécessaire.

Repères utiles

HEAD : position actuelle. origin : nom du remote. origin/main : repère local du distant, actualisé notamment par fetch. .gitignore : règles pour les fichiers non suivis, pas un effacement de l'historique. q : quitter la vue paginée.

Bonus facultatifs

```
git stash push -m "travail en cours" # Met de côté le suivi modifié
git stash list
git stash pop                        # Réapplique ; conflit possible
git tag -a v1.0 -m "Version livrée"
git push origin v1.0
```

Par défaut, stash ne prend pas les fichiers non suivis ; -u les inclut. Rebase, cherry-pick et reflog sont des prolongements, hors évaluation. Ne pas expérimenter une réécriture d'historique partagé sans comprendre ses conséquences.

Documentation : <https://git-scm.com/docs> · <https://docs.github.com>